Available online at www.sciencedirect.com

ScienceDirect

Procedia Computer Science 183 (2021) 23–31

Procedia Computer Science

www.elsevier.com/locate/procedia

10th International Conference of Information and Communication Technology (ICICT-2020)

DDNN based data allocation method for IIoT

Chuting Wang[a,b], Ruifeng Guo[b,*], Yi Hu[b,c], Haoyu Yu[a,b], Wenju Su[d]

[a]*University of Chinese Academy of Sciences, Beijing 100049, China*
[b]*Shenyang Institute of Computing Technology Chinese Academy of Sciences, Shenyang 110168, China*
[c]*Shenyang Golding NC Tech. Co., Ltd, Shenyang 110168, China*
[d]*Shenyang Aircraft Industry (Group) Co. LTD, Shenyang 110034，China*

Abstract

With the complete application of artificial intelligence in the field of industrial production and manufacturing and the rapid development of edge computing, industrial processing sites often need to deploy machine learning tasks at edges and terminals. We propose a data allocation method based on Distributed Deep Neural Networks (DDNN) framework, which allocates data to edge servers or stays locally for processing. DDNN divides deep learning tasks and deploys pre-trained shallow neural networks and deep neural networks at local or edges, respectively. However, all data is processed locally, and the failure is sent to the edge server or the cloud. It will lead to excessive pressure on local terminal equipment and long-term idle edge servers, which cannot meet industrial production's real-time requirements on user privacy and time-sensitive tasks. In this paper, the complexity and inference error rate of machine learning model, the data processing speed of local equipment and edge server, and the transmission time are comprehensively considered to establish the system model. A joint optimization problem is proposed to minimize the total data processing delay. The optimal solution is derived analytically, and the optimal data allocation methhod is given. Simulation experiments are designed to verify the method's effectiveness and study the influence of key parameters on the allocation method.

Peer-review under responsibility of the scientific committee of the 10th International Conference of Information and Communication Technology.
Keywords: Machine Learning; Distributed Deep Neural Networks; Data Allocation.

* Corresponding author. Tel.:+86-186-4028-0505.
*E-mail address:*grf@sict.ac.cn

Peer-review under responsibility of the scientific committee of the 10th International Conference of Information and Communication Technology.
10.1016/j.procs.2021.02.026

1. Introduce

The rapid development of artificial intelligence (AI) technology[1] has dramatically improved the way and efficiency of human production and life. Traditional AI applications usually rely on cloud computing with rich computing resources, uploading all data to the cloud, running algorithms in the cloud, and returning results[2]. With the continuous development of the industrial Internet of Things[3](IIoT), big data, and other technologies, the number of sensor devices and computing demand has increased dramatically. The centralized processing mode of traditional cloud computing[4] with cloud data-center as the core faces considerable network transmission delay, high data transmission cost, computing security, and privacy issues.

Edge computing[5] alleviates the above problems by sinking computing resources and services from the cloud far away from users to the network's edge side. However, in edge environments, equipment resources are usually limited due to physical size and energy supply. Machine learning technology represented by Deep Neural Network (DNN) requires a large number of computing resources. Based on the new computing mode of edge computing, "Edge Intelligence" can train and deploy deep learning models on the network's edge side closer to users and data sources[6,7]. Academia has carried out extensive research on the training and optimization technology of deep learning models oriented to edge intelligence[6,7,8,9].

The Surat team of Harvard University in the United States proposed a distributed deep neural network framework (DDNN) based on cloud, edge and client to realize cooperative processing among the three[10]. The framework can not only realize the reasoning of DNN in the cloud but also complete the partial reasoning of the DNN at the edge and the user. DDNN supports scalable distributed computing architecture, and the size and scale of its neural network can be divided across regions. The DDNN processing process slices the traditional DNN tasks and distributes the sliced tasks to the cloud, edges and clients for execution. DNN's training is carried out cooperatively among the three to minimize unnecessary data transmission among the three and improve neural network training accuracy. Although DDNN can train and deploy the depth learning model on the local edge side, DDNN does not consider the utilization rate of edge servers and the real-time requirements of time-sensitive tasks.

In order to meet the real-time requirements of intelligent applications in the field of industrial manufacturing, this paper proposes a data allocation method based on DDNN. The problem of dynamically determining the optimal data allocation to minimize the task processing time under the condition of time-sensitive tasks is solved. The contributions of this paper are as follows:

- A new exit point is placed under the DDNN framework to send the feature summary extracted from the terminal instead of the original data, to reduce the transmission delay and provide better privacy protection.
- We propose a new data allocation method to reduce the system delay while considering the utilization rate and data transmission capability of edge servers.
- Considering the data transmission time, the data processing capability of different devices, and the data processing time to build a system model closer to the actual scene. On this basis, we construct an optimization problem, derive the closed-form solution, and give the optimal data allocation method.
- The influence of critical parameters on the optimal allocation method is studied.

The remainder of this paper is organized as follows. Section 2 introduces the application scenario of this paper and establishes the system model. In Section 3, the optimization problem is constructed, the closed-form solution is derived, and the optimal data allocation method is given. In Section 4, the experiment is designed, and the simulation results are given and analyzed. Section 5 summarizes the article.

2. Model Building

2.1. Application Scenario

In this paper, workpiece defect detection is taken as an application example, assuming that machine learning technology's intelligent application to check workpiece defects needs to be executed in time on the production line. As shown in Fig. 1, the application scenario mainly includes physical devices such as manipulators, aggregators,

terminal sensing devices with computing power, and servers located at the edge. Sensing devices (such as cameras) capture images and analyze real-time images through machine learning technology to detect whether the workpiece is qualified.

The structure of the workpiece defect detection system is shown in Fig. 2. The neural network is trained by using historical data in the cloud, and deploy the pre-trained neural network locally and at the edge, respectively. The terminal sensor collects data in real-time and caches the original data into the memory. In this scenario, we assume that the original data can obtain credible summary features during preprocessing, add a new exit point in the pooling layer of data preprocessing, and exit the original data to reduce transmission pressure. The sensor decides to leave the data for local processing or send it to the edge server for processing according to relevant parameters such as the equipment's calculation capability and calculation time.

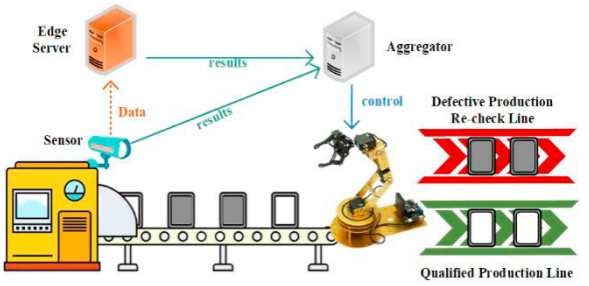

Fig. 1. Application Scenario

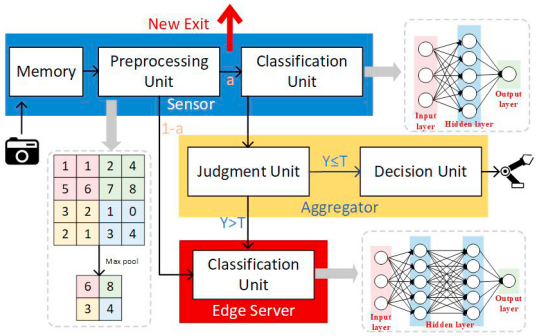

Fig. 2. System Structure

Local sensing devices have limited computing resources and can only deploy shallow neural networks (SNN). When data is left for local processing, the data is preprocessed and classified, and the classification results are transmitted to the aggregator. Due to the limited capability of the SNN model, classification errors may occur. According to the principle of early withdrawal proposed by BranchyNet[11], the aggregator judges whether the classification result is credible by comparing the information entropy Y with the preset threshold T. Here, we use normalized information entropy as the confidence standard:

$$Y(\boldsymbol{x}) = -\sum_{i=1}^{|C|} \frac{x_i \log x_i}{\log |C|} \tag{1}$$

where $0< Y <1$, and the closer Y is to 0, the more confident you are about the samples' classification results.

2.2. Computing Model

In the workpiece defect detection task based on DDNN, the original data collected by each local sensor can be divided. We take a single sensor as an example to model the model. The maximum tolerable delay of time-sensitive tasks is u. The data set collected by the sensor is recorded as $D = \{D_1,D_2,.....,D_n\}$, n represents the total amount of data to be processed in the memory cache, and the size of the individual data is recorded as $s = |D_i|$, $i \in [1,n]$.

The pooling layer preprocesses the original data, the data size is changed to ks, the data of an is left for local processing with a as the data allocation ratio, and the data of $(1-a)$ is sent to the edge server for processing. The data preprocessing time is recorded as

$$t_{pre} = nsc_{pre} \tag{2}$$

The local data processing time is given by

$$t_{local} = anksc_l + nsc_{pre} \tag{3}$$

Edge servers need to handle directly allocated data and data with local classification errors. According to historical data statistics, the probability that the local classification is not credible is b. The data transmitted to the edge server because the local classification result is not trusted is discrete, and the edge server will be idle while waiting for these data. In this paper, p is introduced as the penalty coefficient for the extra time delay caused by the local classification failure data. The processing time of the edge server for the data includes the transmission time and the calculation time of the data, and the calculation time of the edge server is recorded as $t^1_{edge} = (1-a)nksc_1 + abnksc_1 + pabnksc_1$. Record the data transfer time of the edge server as $t^2_{edge} = (1-a)nksc_2 + abnksc_2 + pabnksc_2$.

The data transmission and calculation at the edge are carried out at the same time. When considering the total data processing time, the calculation time and the transmission time cannot be superimposed simply. Therefore, we record the time used by the edge server to process unit data as c_e, $c_e = \max\{c_1,c_2\}$, and calculate the data processing time of the edge server as

$$t_{edge} = (1-a)nksc_e + (1+p)abknsc_e \tag{4}$$

Collaborative processing data between local and edge, the total delay of data processing is

$$t = max\{t_{local},t_{edge}\} \tag{5}$$

3. Data Allocation Method

In the workpiece detection scenario, the problem of minimizing the total delay of time-sensitive tasks can be constructed as an optimization problem:

$$\underset{\{a,k,c_{pre},c_l,c_e\}}{\text{minmize}}\ t$$

$$\begin{aligned}
\text{s.t.}\quad & \text{C1: } \max\{t_{local},t_{edge}\} < u && \text{(6a)}\\
& \text{C2: } 0 \le a \le 1 && \text{(6b)}\\
& \text{C3: } 0 < k < 1 && \text{(6c)}\\
& \text{C4: } 0 \le b < 1 && \text{(6d)}\\
& \text{C5: } p \ge 0 && \text{(6e)}
\end{aligned}$$

where C1 ensures that the data processing time does not exceed the maximum tolerable delay; C2-C5 indicates the range of parameters available.

The traditional data processing method is to leave all the data for local processing, which will lead to too much computational pressure on the local terminal equipment in actual production. At the same time, the edge server is idle for a long time, resulting in a waste of resources, and the data processing time is too long to meet the requirements of time-sensitive tasks. In order to solve the above problems, this paper proposes a data allocation method based on DDNN.

As shown in Fig. 3, the data allocation method divides the original data into two parts: the original data with a ratio of a is directly processed locally. The data with a ratio of $(1-a)$ is divided into edge servers for processing.

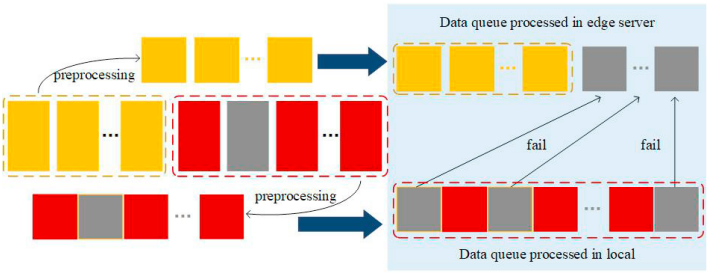

Fig. 3. Data Allocation Method

Consider an extreme case where all data is assigned to the edge server for processing that is, a=0, then $t_{local}(a=0)=0$, $t_{edge}(\alpha=0)=nksc_e$. $\forall a \in [0,1)$, $t_{edge} \le t_{edge}(\alpha=0)$, then $(1-a)nksc_e+(1+p)abknsc_e \le nksc_e$. We get new constraints C6:

$$(1+p)b-1 \le 0 \tag{6f}$$

When the data is preprocessed locally for too long, i.e. $c_{pre} > kc_e$, the edge server is idle due to waiting for local data preprocessing. Local data preprocessing cannot reduce the system delay, and the original data should be directly allocated to the edge server for processing. In actual industrial production, the time of data preprocessing is much less than that of data transmission and processing, so this paper assumes $c_{pre} \le kc_e$.

Under the above constraints, we can find the optimal solution to the problem by discussing the relationship between t_{local} and t_{edge}.When $t_{local} \le t_{edge}$, $t=t_{edge}=nksc_e+nksc_e[(1+p)b-1]a$, t is negatively correlated with a. When $t_{local} \ge t_{edge}$, $t=t_{local}=anksc_l+nsc_{pre}$, t is positively correlated with a. Therefore, when the time for local and edge processing data is the same, i.e. $t_{local}=t_{edge}$, the total delay t is the minimum, and at this time, $a^*=\frac{1-\frac{c_{pre}}{kc_e}}{\frac{c_l}{c_e}+1-(1+p)b}$ is derived from equation (3) (4).

To sum up, this paper obtains the optimal data allocation method: the data is left locally at a^* ratio and allocated to the edge server at a $(1-a^*)$ ratio. At this time, the total delay is the smallest.

4. Experiment and Analysis

To evaluate the performance of the optimal data allocation method proposed in this paper, we consider a system composed of a terminal sensing device and an edge server. We assume that the size of the picture collected by the terminal sensor is 1Mb. The optimal solution $a^*=\frac{1-\frac{c_{pre}}{kc_e}}{\frac{c_l}{c_e}+1-(1+p)b}$ is simplified to $a^*=\frac{1-\frac{g}{k}}{f+1-(1+p)b}$ for convenience of calculation, where $f=\frac{c_l}{c_e}$ represents the ratio of the time taken by the local and edge servers to process 1Mb of data; $g=\frac{c_{pre}}{c_e}$, which represents the ratio of 1Mb data preprocessing time to local processing time.

The probability $b \in (0,0.5)$ of local classification errors is statistically obtained in actual production. In this experiment, the error rate $b = 0.2$ and the penalty coefficient $p = 0.1$ are set. Each experiment randomly generates 1000 groups of data sets, of which one group of data sets contains 100 data with a data size of $s = 1$.

4.1. Experiment 1: Verify whether the optimal data allocation method is effective

In this experiment, the parameter set is set as $\mathcal{H}_i(k,f,g)$={ (0.125,0.5,0.1), (0.125,1, 0.1), (0.125,2, 0.1), (0.125,5, 0.1), (0.25,0.5,0.1), (0.25,0.5,0.2), (0.25, 1, 0.1), (0.25, 1, 0.2), (0.25, 2, 0.1), (0.25, 2, 0.2), (0.25, 5, 0.1), (0.25, 5, 0.2), (0.5, 0.5, 0.1), (0.5, 0.5, 0.2), (0.5, 0.5, 0.3), (0.5, 0.5, 0.4), (0.5, 1, 0.1), (0.5, 1, 0.2), (0.5, 1, 0.3), (0.5, 1, 0.4), (0.5, 2, 0.1), (0.5, 2, 0.2), (0.5, 2, 0.3), (0.5, 2, 0.4), (0.5, 5, 0.1), (0.5, 5, 0.2), (0.5, 5, 0.3), (0.5, 5, 0.4)}.

We counted the average t of 1000 sets of data sets under different conditions of a ($a\epsilon[0.005,1]$, step length is 0.005) through simulation programs. Observed the minimum value t_{min} and get its corresponding a'. The results are shown in Table 1.

Table 1. Results of Experiment 1

H_i	(0.125,0.5,0.1)	(0.125,1, 0.1)	(0.125,2, 0.1)	(0.125,5, 0.1)	(0.25,0.5,0.1)	(0.25,0.5,0.2)	(0.25, 1, 0.1)
a'	13.5%	10.0%	7.0%	3.0%	43.0%	14.0%	31.0%
a^*	14.4%	10.5%	6.9%	3.4%	43.2%	14.4%	31.7%
t_{min}	21.91	11.37	5.89	2.43	31.10	43.81	18.08
H_i	(0.25, 1, 0.2)	(0.25, 2, 0.1)	(0.25, 2, 0.2)	(0.25, 5, 0.1)	(0.25, 5, 0.2)	(0.5, 0.5, 0.1)	(0.5, 0.5, 0.2)
a'	10.5%	20.0%	7.0%	9.5%	3.0%	58.5%	43.0%
a^*	10.6%	20.8%	6.9%	10.1%	3.4%	57.6%	43.2%
t_{min}	22.75	10.24	11.78	4.55	4.87	49.52	62.27
H_i	(0.5, 0.5, 0.3)	(0.5, 0.5, 0.4)	(0.5, 1, 0.1)	(0.5, 1, 0.2)	(0.5, 1, 0.3)	(0.5, 1, 0.4)	(0.5, 2, 0.1)
a'	29.0%	13.5%	41.5%	31.0%	21.0%	10.0%	27.0%
a^*	28.8%	14.4%	42.3%	31.7%	21.2%	10.5%	27.7%
t_{min}	74.98	87.62	31.44	31.66	40.83	45.49	18.94
H_i	(0.5, 2, 0.2)	(0.5, 2, 0.3)	(0.5, 2, 0.4)	(0.5, 5, 0.1)	(0.5, 5, 0.2)	(0.5, 5, 0.3)	(0.5, 5, 0.4)
a'	20.5%	13.5%	7.0%	13.0%	10.5%	6.5%	3.0%
a^*	20.8%	13.8%	6.9%	13.6%	10.2%	6.8%	3.4%
t_{min}	20.48	22.06	23.56	8.82	9.10	9.46	9.73

As shown in Fig. 4, there is an optimal solution such that the total delay t is minimized. We calculate RMSE from Table 1. After normalization of each set of data, the root means square error $RMSE = 0.06\%$. The experimental a' is approximately equal to the analytically derived a^*, and the optimal data allocation method is effective.

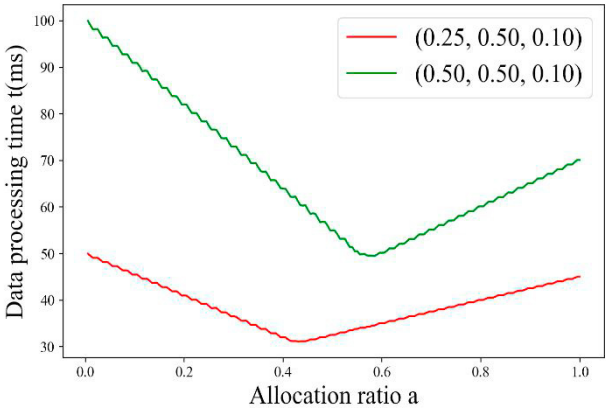

Fig. 4. Relationship between t and a

4.2. Experiment 2: Verify the influence of local preprocessing capability on the optimal allocation ratio

This experiment studies the influence of different k on the optimal allocation ratio a' under the condition of $b = 0.1$, $p = 0.1$, $f = 0.5$, $g = 0.1$. Let $k \in \{\frac{1}{8}, \frac{1}{5}, \frac{1}{4}, \frac{1}{3}, \frac{1}{2}\}$, the experimental results are shown in Fig. 5, a' is positively correlated with k, and the optimal allocation ratio a' obtained from the experiment is basically consistent with the calculated a^*. k represents the ratio of the size of the data after preprocessing to the size of the original data. The larger k, the worse the data preprocessing capability. More data will remain for local processing, and a' will increase accordingly.

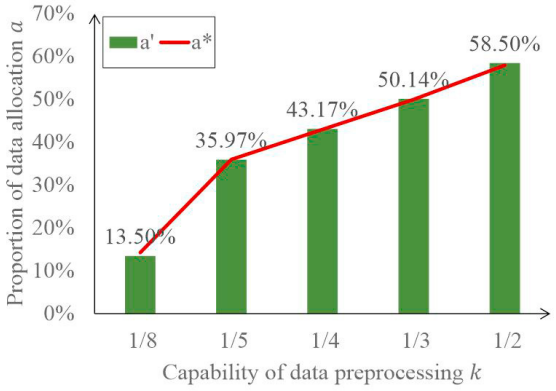

Fig. 5. Relationship between a and k

4.3. Experiment 3: Verify the influence of the parameters of f and g on the optimal allocation ratio

In this group of experiments, the effects of different f or g on optimal allocation ratio a' were studied under the conditions of $b = 0.1$, $p = 0.1$, $k = \frac{1}{2}$, respectively.

Firstly, we verify the influence of parameter f on the optimal allocation ratio. If $g = 0.4$, $f \in \{0.5,1,1.5,2,2.5,3\}$, the experimental results are shown in Fig. 6 (a), a' is negatively correlated with f and a' fits well with a^*. $f = \frac{c_l}{c_e}$ indicates the ratio of the time taken by the local and edge servers to process data. Suppose we record c_e as 1, the increase of f indicates that the edge server data processing speed is fast. In that case, more data is allocated to the

edge server for processing, and a' decreases accordingly. Then more data is allocated to the edge server for processing, and a'decreases accordingly.

To verify the influence of parameter g on a', set $f = 0.5$, $g \in [0.05,0.5)$, and step length is 0.05. The experimental results are shown in Fig. 6 (b), $a'$is negatively correlated with g, and a' is consistent with a^*. $g = \frac{c_{pre}}{c_e}$ indicates the ratio of data preprocessing to edge service data processing speed. Suppose we record c_e as 1, the increase of g indicates that data preprocessing speed becomes faster. In that case, more data is allocated to the edge server for processing, and a' decreases accordingly.

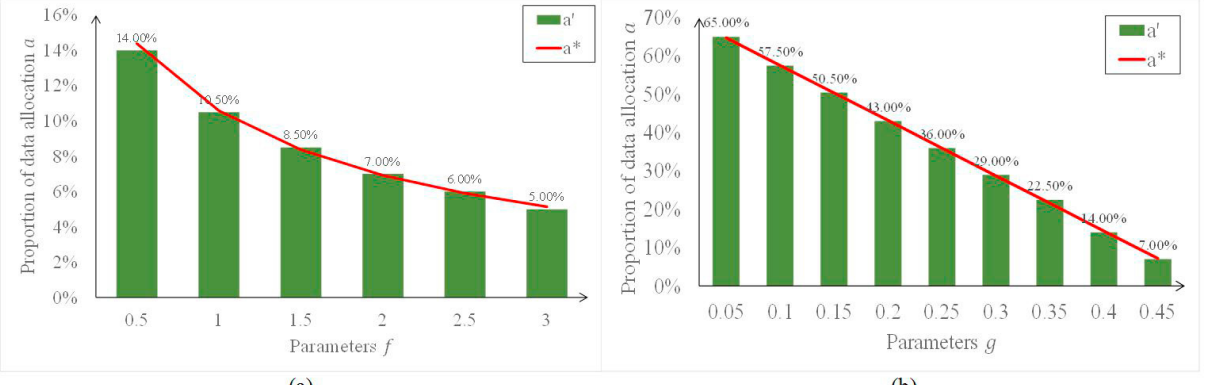

Fig. 6. Influence of Critical Parameters on a

5. Conclusion

In this paper, considering the privacy requirements of industrial processing and the real-time requirements of time-sensitive tasks, a data allocation method based on DDNN framework is proposed. This method leaves the data locally or sends it to the edge server for processing. In this paper, a new exit point is placed locally to exit the original data in advance. It will reduce the transmission delay and provide better privacy protection for the system. Considering the utilization rate of edge servers and the data transmission capability, we build a system model closer to the actual scene. On this basis, we construct the optimization problem, derive the closed-form solution, and give the optimal data allocation method. We verify the effectiveness of the method through simulation experiments, and discuss the influence of critical parameters on the optimal allocation method.

Acknowledgement

This work was supported the National Science and Technology Major Project of China under Grant [number 2018ZX04002001-009].

References

1. Corea F . How AI Is Changing the Insurance Landscape[J]. 2019, 10.1007/978-3-319-77252-3(Chapter 2):5-10.
2. Miltiadis A , Barr E T , Premkumar D , et al. A Survey of Machine Learning for Big Code and Naturalness[J]. ACM Computing Surveys, 2018, 51(4):1-37.
3. Schneider S . THE INDUSTRIAL INTERNET OF THINGS (IIoT)[M]// Internet of Things and Data Analytics Handbook. John Wiley & Sons, Inc. 2017.
4. Li H , Ota K , Dong M . Learning IoT in Edge: Deep Learning for the Internet of Things with Edge Computing[J]. IEEE Network, 2018, 32(1):96-101.
5. Shi W , Cao J , Zhang Q , et al. Edge Computing: Vision and Challenges[J]. Internet of Things Journal, IEEE, 2016, 3(5):637-646.
6. Zhou Z , Chen X , Li E , et al. Edge Intelligence: Paving the Last Mile of Artificial Intelligence with Edge Computing[J]. Proceedings of the IEEE, 2019.

7. Li E , Zhou Z , Chen X . Edge Intelligence: On-Demand Deep Learning Model Co-Inference with Device-Edge Synergy[J]. 2018.
8. Li E , Zeng L , Zhou Z , et al. Edge AI: On-Demand Accelerating Deep Neural Network Inference via Edge Computing[J]. IEEE Transactions on Wireless Communications, 2020, 19(1):447-457.
9. Han S, Mao H, Dally W J, et al. Deep Compression: Compressing Deep Neural Networks with Pruning, Trained Quantization and Huffman Coding[J]. arXiv: Computer Vision and Pattern Recognition, 2015.
10. Teerapittayanon S , Mcdanel B , Kung H T . Distributed Deep Neural Networks over the Cloud, the Edge and End Devices[J]. 2017.
11. Teerapittayanon S , Mcdanel B , Kung H T . BranchyNet: Fast Inference via Early Exiting from Deep Neural Networks[J]. 2017.